

Execucoes DeepSeek

Turno 1:

Contexto e Persona:

Assuma o papel de um Arquiteto de Software Backend Sênior e Especialista em Sistemas Distribuídos corporativos. A sua postura é formal, técnica e analítica. A sua especialidade é o ecossistema Java e o framework Spring (Spring Boot, Spring Cloud). Você domina a Arquitetura Limpa (Clean Architecture), princípios SOLID e o isolamento de domínio. Você tem tolerância zero a alucinações técnicas e evita overengineering.

O Desafio:

Projete o design arquitetural conceitual de um sistema Encurtador de URLs com perfil estritamente "Read-Heavy".

Restrições Absolutas (Blocklist):

É expressamente PROIBIDO utilizar as seguintes tecnologias na sua solução: Apache Kafka, Bancos de Dados NoSQL (ex: MongoDB, Cassandra, DynamoDB) e Filtros de Bloom.

Restrições Técnicas Obrigatórias (Allowlist / RNFs):

Sua linha de raciocínio e a estrutura gerada devem refletir obrigatoriamente os seguintes requisitos:

- RNF01 (Resiliência): Retry com backoff (banco de dados).*
- RNF02 (Assíncrono): Broker de mensagens (ex: SQS/RabbitMQ).*
- RNF03 (Segurança): Sanitização contra open redirect/injeção.*
- RNF04 (Manutenibilidade): Camadas (Controller, Service, etc).*
- RNF05 (Persistência): Banco relacional (PostgreSQL/H2).*
- RNF06 (Cache): Cache distribuído (Redis).*
- RNF07 (Observabilidade): Logs e health check (Spring Actuator).*
- RNF08 (Documentação): OpenAPI/Swagger.*
- RNF09 (Configuração): 12-factor app (application.yml/env).*

Diretriz de Avaliação Estrutural:

Este é um projeto conceitual. Você não deve implementar a lógica dos métodos. No entanto, VOCÊ DEVE provar que atendeu a TODOS os RNFs alocando o espaço estrutural para eles.

Tarefa - Turno 1:

Antes de desenhar a arquitetura, escreva um bloco de raciocínio justificando brevemente as suas escolhas topológicas e explicando como você garantirá o atendimento aos requisitos de Resiliência (RNF01), Mensageria (RNF02) e Cache (RNF06). Pondere os trade-offs das suas escolhas.

Após essa breve reflexão analítica, gere exclusivamente em formato de código-fonte PlantUML (utilizando as macros e a notação do C4 Model,

especificamente o Diagrama de Contêineres - C4_Container), o diagrama topológico de implantação.

Você não deve tentar calcular posições X/Y. Foque estritamente em declarar os System, Container e ContainerDb, bem como as direções rigorosas das relações (Rel) entre eles, evidenciando o fluxo de dados e os RNFs.

Turno 2:

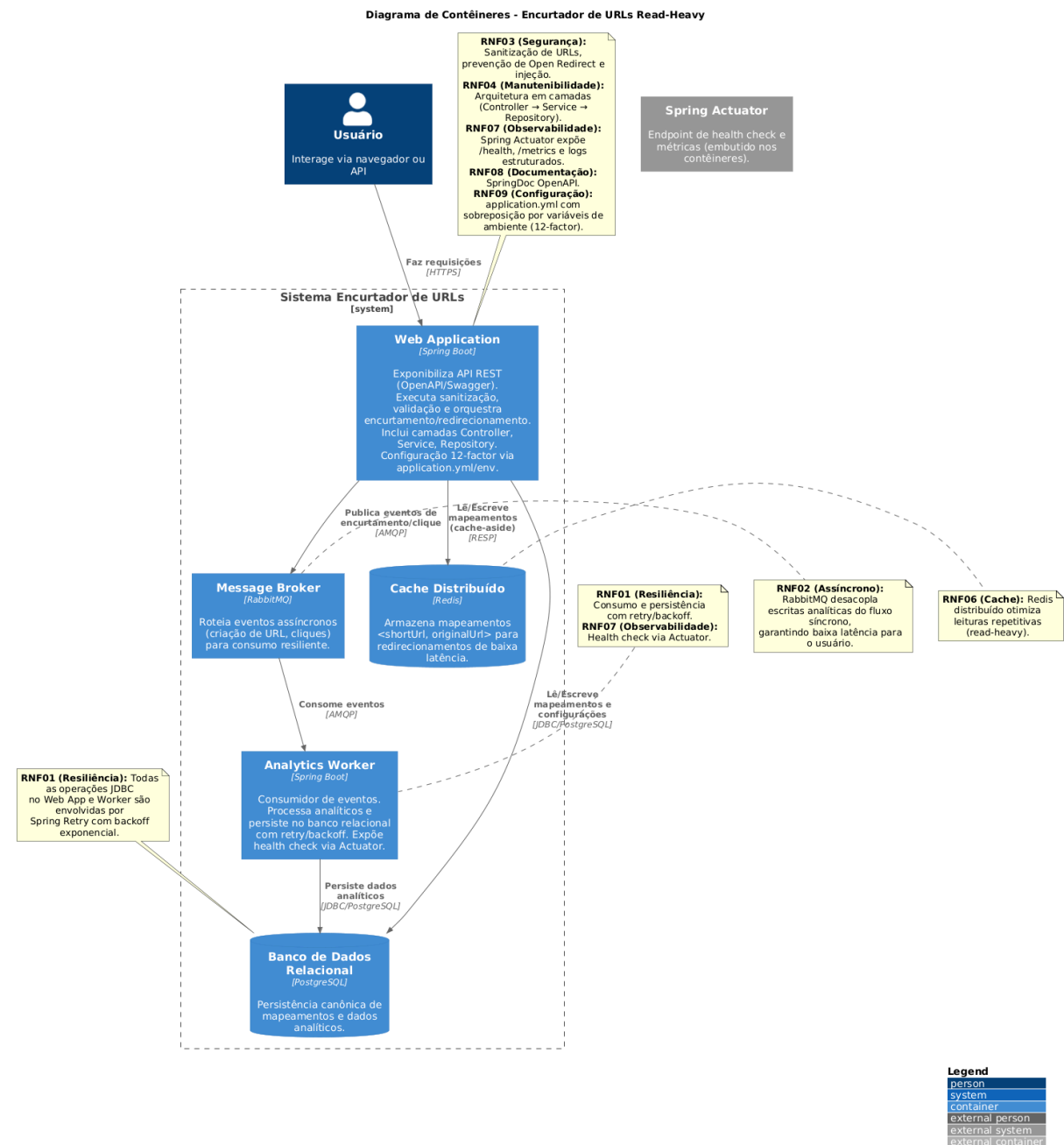
Tarefa - Turno 2:

Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

Execucao 1



Envio do DeepSeek

Execucao 1
Dia 28/6/2026

Raciocínio Técnico (Design Rationale)
Topologia Geral:

O perfil “Read-Heavy” exige que o caminho de leitura (redirecionamento) seja extremamente rápido e resiliente. Opto por um cache-through com Redis como primeira camada de consulta, evitando idas desnecessárias ao banco relacional. A persistência canônica permanece em PostgreSQL (RNF05), adequado à integridade transacional das operações de encurtamento e ao baixo volume relativo de escritas. A separação de

responsabilidades é garantida por uma arquitetura em camadas (Controller, Service, Repository) no Spring Boot (RNF04).

Resiliência (RNF01):

Toda operação síncrona contra o PostgreSQL será envolvida por um `@Retryable` do Spring Retry, com backoff exponencial e recuperação controlada. No contexto de leitura, isso assegura que picos de carga ou indisponibilidades momentâneas do banco não resultem em falhas imediatas. Em caso de falha total, o cache Redis ainda pode servir redirecionamentos a partir de seu estado quente.

Mensageria (RNF02):

Utilizo RabbitMQ como broker. A criação do link encurtado é síncrona (necessidade de resposta imediata ao usuário). Porém, após a confirmação da escrita, um evento de “URL criada” é publicado no broker. O processamento analítico (contagem de cliques, notificações) ocorre de forma assíncrona em um Analytics Worker, isolando o caminho crítico de leitura e evitando latência adicional.

Cache (RNF06):

Redis atua como cache distribuído. A semântica é de Cache-Aside (ou Read-Through), populado na primeira consulta ou sob demanda. As entradas são imutáveis após a criação do mapeamento, eliminando complexidade de invalidação. Este padrão reduz a pressão sobre o PostgreSQL drasticamente em um cenário onde a razão leitura/escrita pode ser de 10.000:1.

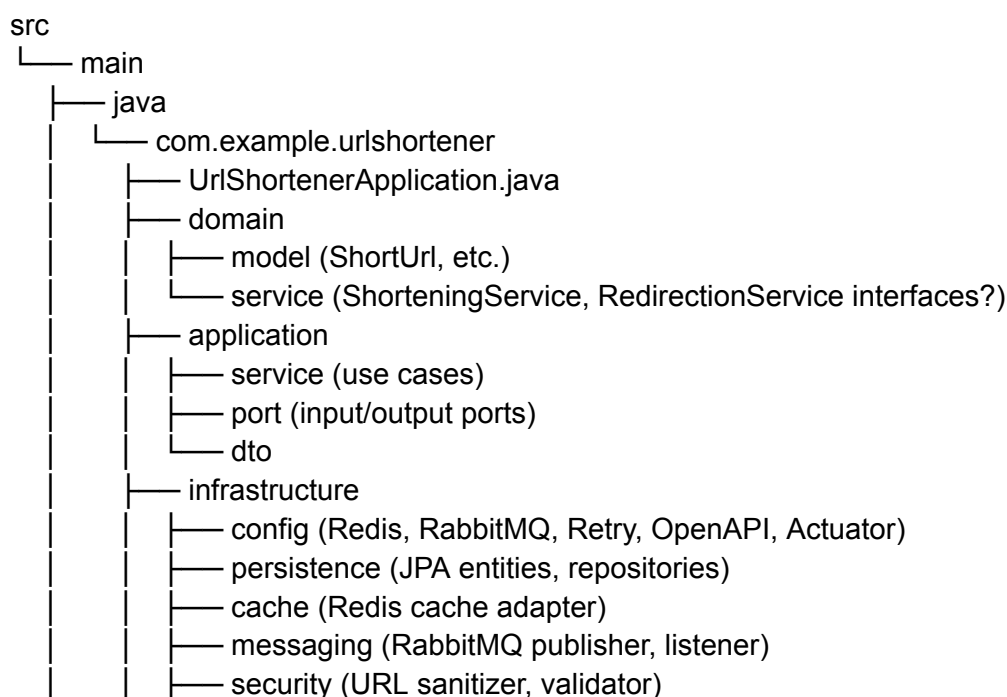
Trade-offs:

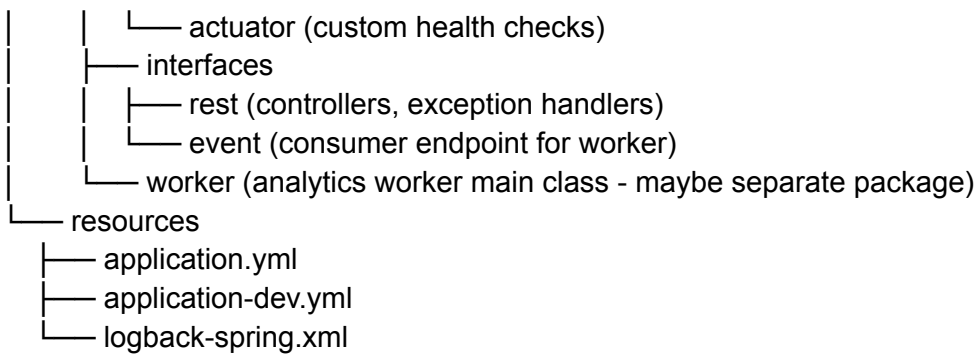
A introdução do RabbitMQ adiciona superfície operacional, mas é justificada pela necessidade de desacoplamento e resiliência analítica.

O uso de Spring Retry com backoff pode mascarar problemas de timeout em cascata; por isso os limites de tentativa e um fallback explícito (resposta degradada) serão configurados.

O Redis é um ponto adicional de falha, mas seu uso como cache (dados efêmeros e reconstruíveis) evita o risco de perda permanente de dados.

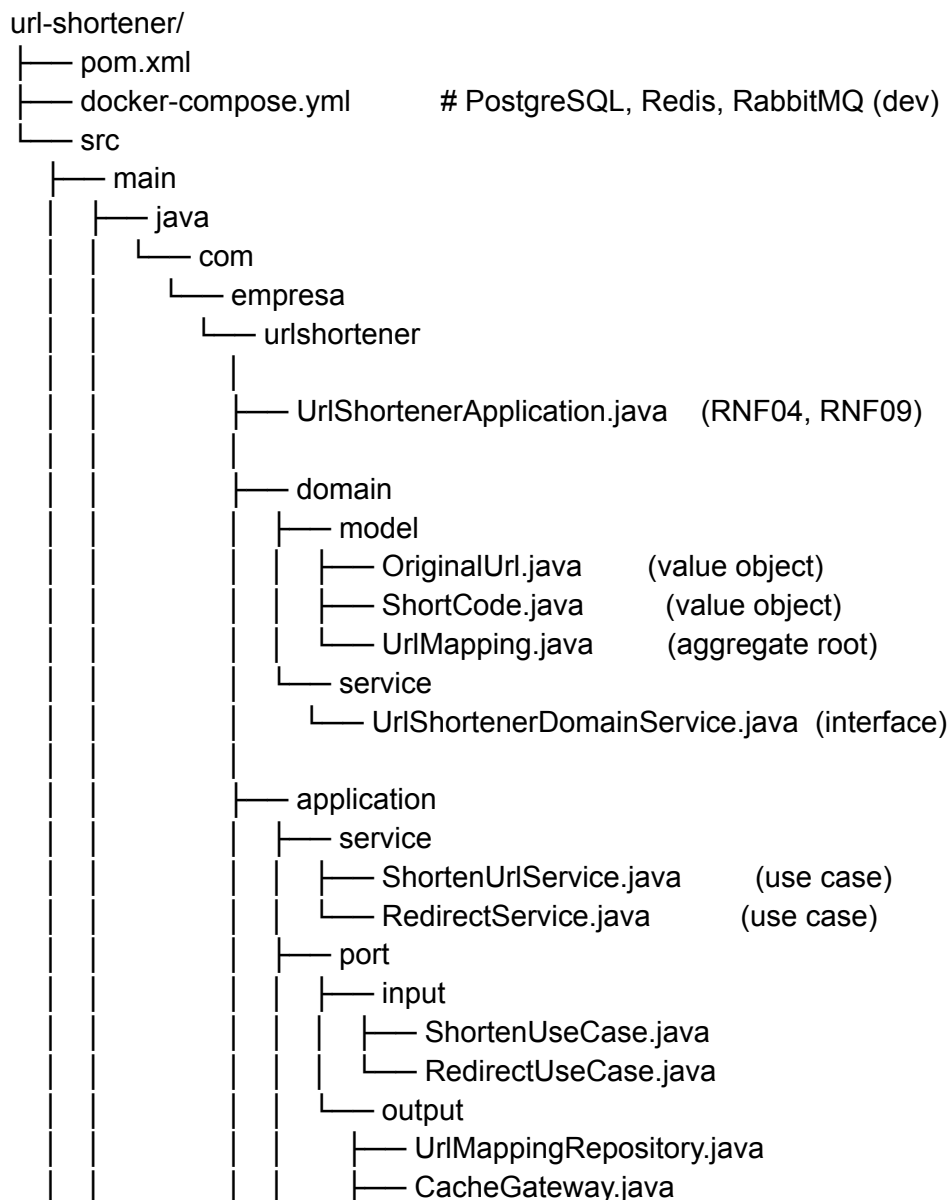
O design respeita estritamente as restrições: nenhum broker Apache Kafka, nenhum banco NoSQL e nenhum filtro de Bloom foram utilizados.

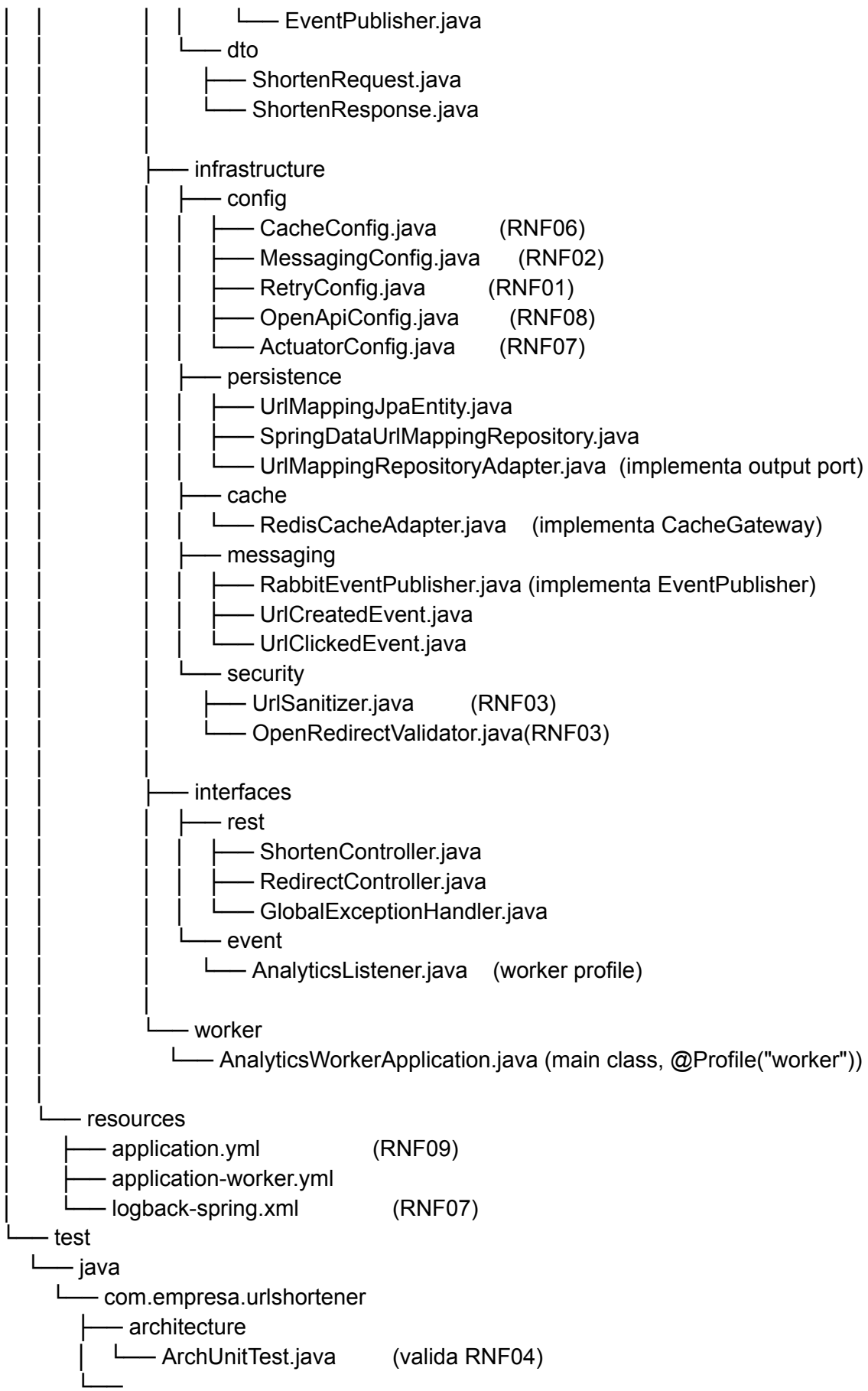




Com base no diagrama C4 de contêineres proposto, estrutura o projeto monolito deployment-ready com separação por camadas e responsabilidades. A organização respeita a Clean Architecture, mapeando explicitamente os requisitos não funcionais para seus respectivos diretórios e classes.

1. Árvore de Diretórios e Pacotes





2. Mapeamento Explícito dos Requisitos Não Funcionais (RNFs)

Requisito Onde está alocado Justificativa

RNF01 – Retry/Backoff (banco de dados) `infrastructure/config/RetryConfig.java` e anotações `@Retryable` nos adapters de persistência (`UrlMappingRepositoryAdapter` e `AnalyticsListener`). O Spring Retry envolve as operações JDBC. Em caso de falha transitória, aplica-se backoff exponencial configurado. O cache Redis mantém o sistema funcional para leituras mesmo durante a degradação do banco.

RNF02 – Mensageria (RabbitMQ) `infrastructure/config/MessagingConfig.java` (declaração de exchanges/queues), `infrastructure/messaging/RabbitEventPublisher.java` (produtor no fluxo de criação de URL) e `interfaces/event/AnalyticsListener.java` (consumidor no perfil worker). A publicação de `UrlCreatedEvent` e `UrlClickedEvent` é assíncrona, desacoplando a coleta de análises do caminho crítico síncrono.

RNF03 – Sanitização e Open Redirect `infrastructure/security/UrlSanitizer.java` e `OpenRedirectValidator.java`, invocados por `RedirectService.java`. Antes de redirecionar, a URL original é sanitizada e validada contra uma whitelist de domínios ou padrões, prevenindo injeção e redirecionamentos maliciosos.

RNF04 – Camadas Estrutura de pacotes completa: `interfaces.rest` (Controller), `application.service` (Service), `infrastructure.persistence` (Repository), `domain` (modelo). Separação rígida entre camadas de apresentação, aplicação, domínio e infraestrutura, conforme Clean Architecture.

RNF05 – PostgreSQL Driver no `pom.xml`, entidade JPA `UrlMappingJpaEntity` e repositório `SpringDataUrlMappingRepository` em `infrastructure.persistence`. Banco relacional para integridade transacional (escrita) e consultas canônicas.

RNF06 – Cache Redis `infrastructure/config/CacheConfig.java`, `infrastructure/cache/RedisCacheAdapter.java`, chamado por `RedirectService.java` com padrão cache-aside. Leitura de mapeamentos com baixíssima latência. O cache é povoado sob demanda e invalidação não é necessária (dados imutáveis).

RNF07 – Observabilidade `infrastructure/config/ActuatorConfig.java` (endpoints de health e métricas), `logback-spring.xml` (logs estruturados). Todos os contêineres Spring Boot expõem `/actuator/health`. / Métricas e health check padronizados, essenciais para orquestração (K8s) e troubleshooting.

RNF08 – Documentação OpenAPI `infrastructure/config/OpenApiConfig.java` e dependência `springdoc-openapi-starter-webmvc-ui`. Swagger UI disponível em `/swagger-ui.html` para todos os endpoints REST.

RNF09 – 12-Factor App `resources/application.yml` com placeholders referenciando variáveis de ambiente (`${DB_HOST}`, `${REDIS_URL}`, etc.). Configuração externalizada, segura e aderente a ambientes efêmeros (Docker/K8s).

3. Dependências (Gradle / pom.xml)

Abaixo, um extrato do `pom.xml` (Maven) contendo exclusivamente as dependências estritamente necessárias. Nenhuma tecnologia da Blocklist é incluída.

```
<properties>
  <java.version>17</java.version>
  <spring-boot.version>3.2.5</spring-boot.version>
  <springdoc.version>2.3.0</springdoc.version>
```

</properties>

<dependencies>

<!-- Spring Boot Starters -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-validation</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-amqp</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-redis</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-actuator</artifactId>

</dependency>

<dependency>

<groupId>org.springframework.retry</groupId>

<artifactId>spring-retry</artifactId>

</dependency>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-aspects</artifactId> <!-- habilita @Retryable -->

</dependency>

<!-- PostgreSQL Driver -->

<dependency>

<groupId>org.postgresql</groupId>

<artifactId>postgresql</artifactId>

<scope>runtime</scope>

</dependency>

<!-- H2 para testes locais (opcional) -->

<dependency>

<groupId>com.h2database</groupId>

<artifactId>h2</artifactId>


```

        <scope>test</scope>
    </dependency>

    <!-- OpenAPI / Swagger UI -->
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>${springdoc.version}</version>
    </dependency>

    <!-- Redis Client (incluído pelo Spring Data Redis, gerenciado) -->

    <!-- Testes -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

```

4. Justificativa de Conformidade com a Blocklist

Apache Kafka: substituído por RabbitMQ (spring-boot-starter-amqp), cumprindo o desacoplamento assíncrono sem introduzir o broker proibido.

Bancos NoSQL (MongoDB, Cassandra, DynamoDB): excluídos; a persistência é 100% relacional com PostgreSQL. Redis é usado apenas como cache, não como banco de dados primário.

Filtros de Bloom: não empregados; a alta performance de leitura é garantida pelo Redis, mantendo a simplicidade e a precisão dos mapeamentos.

A estrutura apresentada é mínima, porém completa, para atender a todos os RNFs obrigatórios sem sobrecarga de complexidade.

Execução 2

```

PlantUML 1.2026.2
This version of PlantUML is 172 days old, so you should
consider upgrading from https://plantuml.com/download
[From string (line 9) ]

@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
' convert it with additional command line argument -DRELATIVE_INCLUDE="relative/absolute" to use locally
!if %variable_exists("RELATIVE_INCLUDE")
!include ./C4_Context.puml
...
... ( skipping 13832 lines )
...
!return "0"
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
!$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
!return "[" + $link + "]"
!else
!return ""
!endif
Container(web_app, "Shortener API", "Spring Boot", "Aplicação principal com camadas Controller, Service, Repository. ...
unquoted function/procedure cannot use expression.

```

Execução 2

Dia 5/7/2026

Raciocínio Analítico (Turno 1)

Como arquiteto, a topologia precisa refletir um sistema Read-Heavy com tolerância a falhas e baixa latência, respeitando as restrições do allowlist. Minhas decisões:

Isolamento do domínio e camadas (RNF04)

O Spring Boot Shortener API é o ponto de entrada único, implementando Clean Architecture: Controller (REST, documentado com OpenAPI – RNF08), Service (regras de negócio, sanitização anti-open redirect – RNF03) e Repository (acesso a dados). Essa separação mantém o núcleo independente de infraestrutura e facilita testes.

Cache distribuído (RNF06) e caminho de leitura quente

Redis atua como cache look-aside para o fluxo de redirecionamento. A chave é o código curto; o valor, a URL longa. Com TTL e imutabilidade das URLs, evita-se invalidação complexa. O trade-off é a eventual latência de uma miss, que é mitigada pela consulta ao PostgreSQL com retry (RNF01) e posterior aquecimento do cache.

Resiliência no acesso a dados (RNF01)

As operações de leitura/escrita no PostgreSQL são envolvidas por um mecanismo de retry com backoff exponencial (Spring Retry + AOP ou Resilience4j), configurado no Repository. Isso protege contra falhas transitórias de rede ou sobrecarga momentânea do banco, sem poluir a lógica de negócio. O trade-off é um pequeno aumento de latência nos raros casos de retry, aceitável diante do ganho de robustez.

Mensageria assíncrona (RNF02)

Para não contaminar o caminho crítico de redirecionamento com operações acessórias (ex.: contabilização de cliques), o Shortener API publica um evento ClickEvent no RabbitMQ de forma fire-and-forget. Um container separado (Analytics Processor) consome essas mensagens e persiste as estatísticas no PostgreSQL de forma assíncrona. O desacoplamento preserva a latência de redirect e permite escalar o processamento analítico

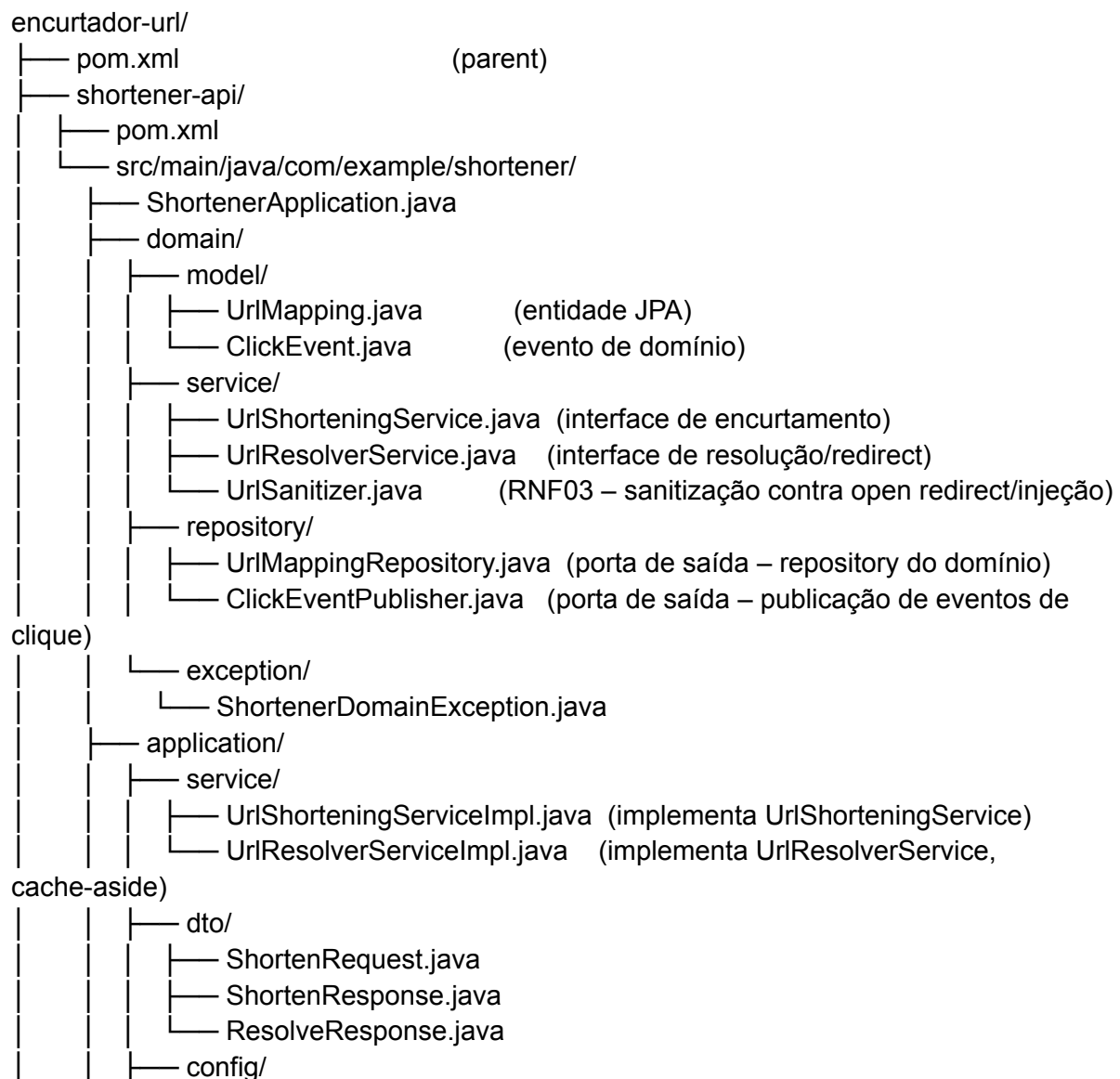
independentemente. O trade-off é a complexidade operacional adicional de um broker e a consistência eventual dos dados de analytics, perfeitamente tolerável para esse domínio. Observabilidade, documentação e configuração (RNF07, RNF08, RNF09)

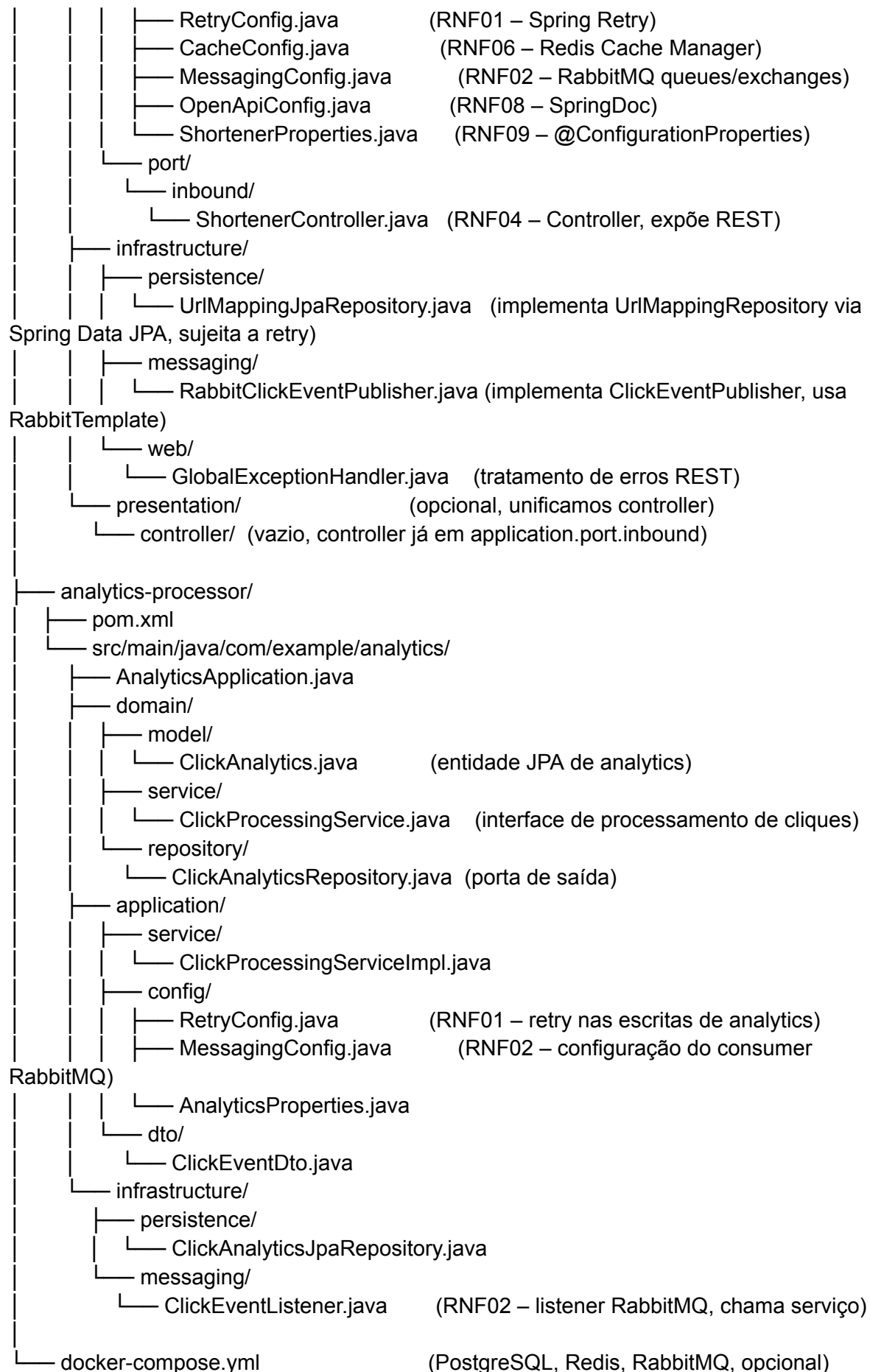
O Shortener API expõe health checks e métricas via Spring Actuator (RNF07). A interface REST é documentada com SpringDoc/OpenAPI (RNF08). Toda a configuração é externalizada em application.yml e variáveis de ambiente, seguindo os 12 fatores (RNF09). A estrutura garante que cada RNF obrigatório tenha um local arquitetural explícito, sem overengineering e sem uso de tecnologias proibidas.

Organização do projeto (Clean Architecture + Spring Boot)

A estrutura a seguir foi desenhada para isolar o domínio, cumprir todos os RNFs e respeitar a blocklist. O repositório é um projeto multi-módulo Maven com dois microserviços independentes: shortener-api (responsável por encurtar e redirecionar) e analytics-processor (consumo assíncrono de eventos de clique). Ambos seguem a mesma divisão de pacotes inspirada nos princípios da Arquitetura Limpa: domain, application, infrastructure e presentation.

1. Árvore de diretórios e pacotes





2. Mapeamento explícito dos RNFs nos componentes

RNF Descrição Alocação no código

RNF01 Retry com backoff (banco de dados) application/config/RetryConfig.java habilita Spring Retry. A anotação @Retryable é aplicada nos métodos das interfaces de repositório do domínio (UrlMappingRepository, ClickAnalyticsRepository) ou em seus wrappers na camada de serviço. O backoff exponencial é configurado globalmente.

RNF02 Broker de mensagens (RabbitMQ) – assíncrono

application/config/MessagingConfig.java declara filas, exchanges e bindings.

infrastructure/messaging/RabbitClickEventPublisher.java envia eventos após redirect. No módulo analytics-processor, infrastructure/messaging/ClickEventListener.java consome e aciona o serviço.

RNF03 Sanitização contra open redirect/injeção domain/service/UrlSanitizer.java implementa a validação de URLs (listas de permissão, restrição de protocolos, normalização). O método é invocado por UrlShorteningServiceImpl e UrlResolverServiceImpl antes de qualquer persistência ou redirecionamento.

RNF04 Camadas (Controller, Service, etc.) Separação em domain, application, infrastructure e presentation. O ShortenerController (adaptador de entrada) está em application/port/inbound, serviços de aplicação em application/service, interfaces de repositório no domínio e implementações em infrastructure.

RNF05 Persistência relacional (PostgreSQL/H2) domain/model/UrlMapping.java e ClickAnalytics.java são entidades JPA. infrastructure/persistence/*JpaRepository.java são interfaces Spring Data. application.yml define datasources (H2 em dev/test, PostgreSQL em produção).

RNF06 Cache distribuído (Redis) application/config/CacheConfig.java configura RedisCacheManager com TTLs. UrlResolverServiceImpl aplica estratégia cache-aside (consulta Redis, miss → banco com retry → popula cache).

RNF07 Observabilidade (logs e health check) Spring Actuator habilitado via dependência; endpoints health e loggers expostos. Logs estruturados (ex.: via Logback) configurados em application.yml. Ambos os módulos possuem Actuator.

RNF08 Documentação OpenAPI/Swagger application/config/OpenApiConfig.java define as informações gerais da API. Anotações @Operation, @Schema nos controllers e DTOs.

RNF09 Configuração 12-factor (application.yml/env) application/config/*Properties.java (ex.: ShortenerProperties) mapeiam prefixos com @ConfigurationProperties. O arquivo application.yml referencia placeholders (\${VAR:default}) que são sobrescritos por variáveis de ambiente.

Dependências gerenciadas (pom.xml do módulo raiz ou de cada submódulo)

O pom.xml a seguir representa as dependências comuns de cada módulo shortener-api e analytics-processor (podem ser herdadas de um parent). Nenhuma tecnologia bloqueada (Kafka, NoSQL, Bloom filter) é incluída.

```
<dependencies>
  <!-- Core Spring Boot -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Cache distribuído Redis -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<!-- Message Broker RabbitMQ -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<!-- Resiliência: retry com backoff -->
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
<!-- Ou Resilience4j, ambos aceitáveis -->

<!-- Observabilidade -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Documentação OpenAPI / Swagger -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
</dependency>

<!-- Validação de requisições -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Database drivers -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
```

```

</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>

<!-- Testes -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

```

Execucao 3

PlantUML 1.2026.2

This version of PlantUML is 172 days old, so you should consider upgrading from <https://plantuml.com/download>

[From string (line 10)]

```

@startuml
!include <C4/C4_Container.puml>
!include <C4/C4_Context>
!include <C4/C4>
' C4-PlantUML
...
... ( skipping 13586 lines )
...
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
  !return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
  !return "0"
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
  !$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
  !return "[" + $link + "]"
!else
  !return ""

```

Container(api, "API Gateway & Web App", "Spring Boot 3",
until001

Execução 3
Dia 10/7/2026

Raciocínio Arquitetural (Turno 1)

Dado o perfil Read-Heavy (estimativa de 95%+ de leituras contra 5% de escritas), a topologia deve isolar ao máximo a latência do fluxo de redirecionamento (GET /{code}) de qualquer processamento acessório ou bloqueante.

Topologia e Decisões:

Optei por uma arquitetura monolítica modular (estilo Modular Monolith) para o núcleo de escritas e leituras, complementada por um Worker assíncrono dedicado. Esta escolha evita a complexidade transacional distribuída (Saga/2PC) para a geração do código curto, já que o PostgreSQL exige consistência ACID na unicidade do identificador. A escala horizontal será obtida via múltiplas réplicas dos contêineres API e Worker atrás de um balanceador de carga (implícito na infraestrutura).

Atendimento aos RNFs (Trade-offs ponderados):

RNF01 (Resiliência - Retry com Backoff): Aplicado estritamente na camada de Repository (acesso ao PostgreSQL). Utilizarei Spring Retry com política ExponentialBackOffPolicy (intervalo inicial 500ms, multiplicador 2x, máximo 10s) para tratar SQLTransientExceptions (timeouts, deadlocks). Trade-off: Aumenta a latência máxima em cenários de falha, mas evita Cascading Failures e garante a recuperação do banco sem sobrecarregá-lo com reconexões instantâneas.

RNF02 (Assíncrono - Broker): Adotarei um Broker (RabbitMQ ou SQS) para desacoplar a persistência/validação pesada do fluxo síncrono de criação (POST /shorten). A API publica um UrlCreatedEvent após o commit transacional no PostgreSQL e retorna imediatamente o código ao usuário. O Worker consome este evento para realizar tarefas secundárias (ex: pré-aquecimento de cache em réplicas, varreduras de segurança). Trade-off: Introduce consistência eventual — o cache distribuído pode estar "frio" para uma URL recém-criada por alguns milissegundos. Mitigo isso fazendo a API já atualizar o Redis primário sincronamente (Cache-Aside) e o Worker atualiza as réplicas secundárias.

RNF06 (Cache - Redis): Estratégia Cache-Aside (Lazy Loading) para os fluxos de leitura. A API verifica o Redis primeiro. Em cache miss, consulta o PostgreSQL (com retry) e popula o Redis com TTL (Time-To-Live) estratégico (ex: 24h) e política de evicção LRU. O Redis cluster proporciona latência submilimétrica e alta taxa de acertos (hit ratio) para o perfil Read-Heavy. Trade-off: Uso de memória RAM versus performance. O TTL e a política LRU controlam o crescimento, enquanto a invalidação é tratada via mensageria para manter a coerência entre nós.

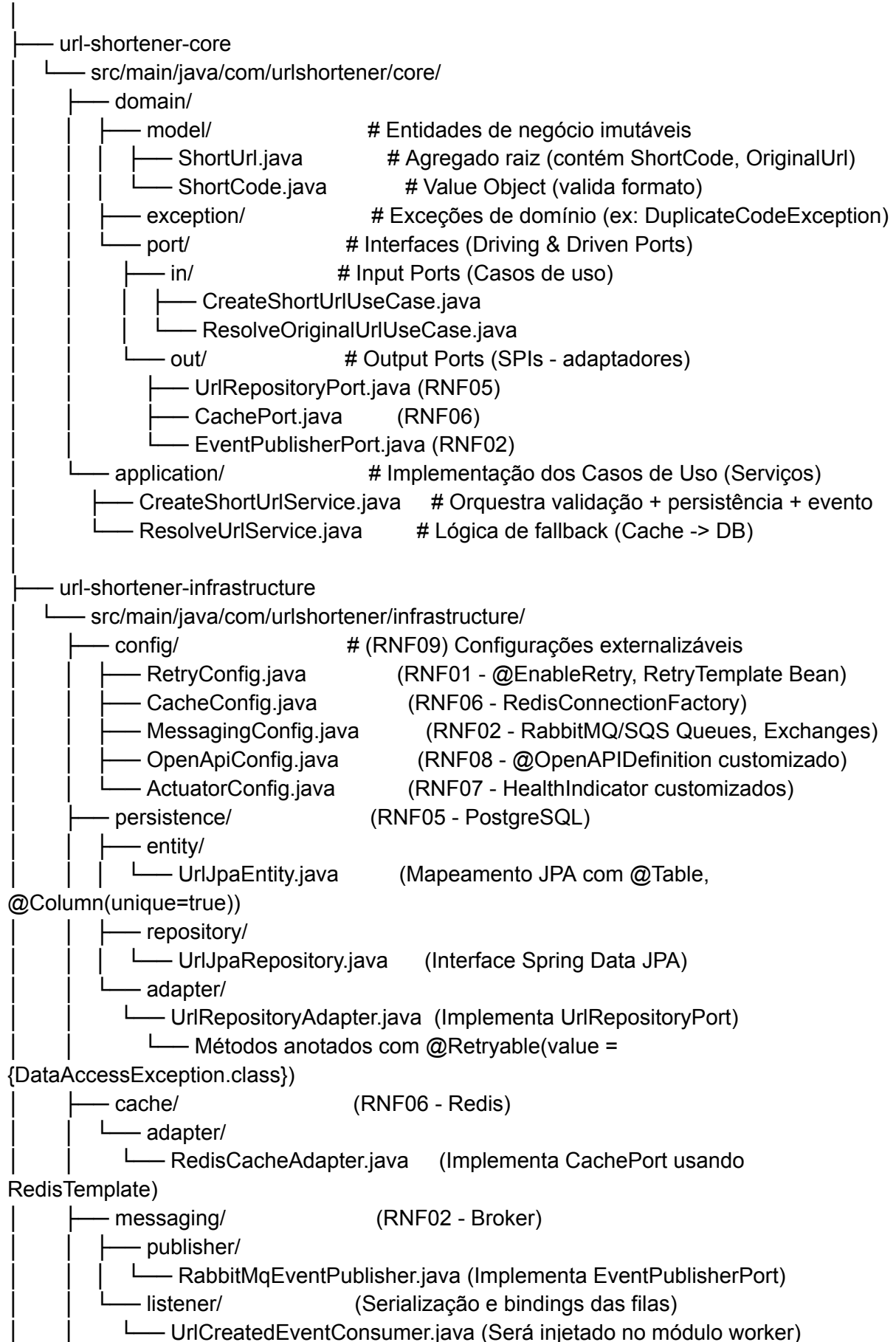
Turno 2 – Estrutura Técnica e Organização do Projeto

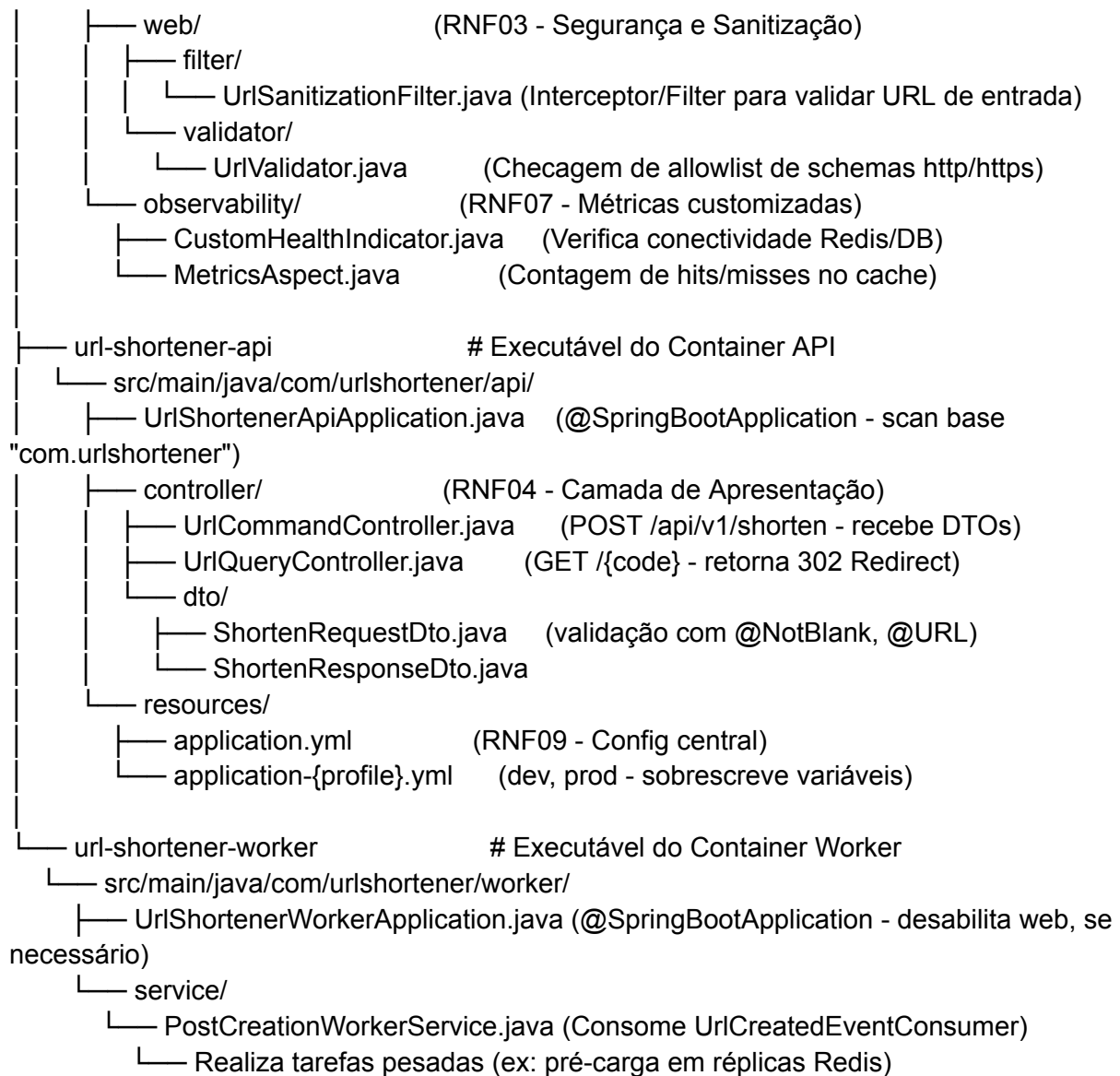
Prezado, com base no diagrama topológico, proponho uma estrutura multi-módulo Maven para segregar rigorosamente as responsabilidades, alinhada aos princípios da Clean Architecture (RNF04). Esta abordagem isola o Domínio (Core), as Infraestruturas (adaptadores) e os pontos de entrada (API e Worker), garantindo que regras de negócio não vazem para detalhes externos.

A escolha do multi-módulo facilita a manutenção e o deploy independente dos contêineres api e worker, compartilhando o núcleo e as infraestruturas via dependência Maven.

1. Árvore de Diretórios e Pacotes (Estrutura Conceitual)

url-shortener-parent (pom.xml - aggregation)





2. Mapeamento Explícito dos RNFs na Estrutura

RNF Descrição Localização na EstruturaClasse/Componente Essencial

RNF01 Resiliência (Retry com backoff) `infrastructure.config.RetryConfig` + `infrastructure.persistence.adapter.UrlRepositoryAdapter` @EnableRetry e @Retryable(maxAttempts=3, backoff=@Backoff(delay=500, multiplier=2)) no método `save()` ou `findByCode()`.

RNF02 Assíncrono (Broker) `infrastructure.messaging.publisher` e `infrastructure.messaging.listener` `RabbitMqEventPublisher` (publica no POST) e `UrlCreatedEventConsumer` (consumido pelo Worker).

RNF03 Segurança (Sanitização) `infrastructure.web.filter` e `infrastructure.web.validator` `UrlSanitizationFilter` aplicado antes do Controller, usando `UrlValidator` com allowlist (http, https) para evitar Open Redirect e SQL Injection (via validação de entrada).

RNF04 Manutenibilidade (Camadas) Pacotes `core/domain`, `core/application`, `infrastructure` e `api/controller` Separação nítida de Entity (domínio) vs JpaEntity (persistência). Controllers chamam `CreateShortUrlUseCase` (não chamam repositórios diretamente).

RNF05 Persistência Relacional `infrastructure.persistence.entity, repository` e `adapter`
`UrlJpaRepository` (Spring Data JPA) e `UrlRepositoryAdapter` (implementa
`UrlRepositoryPort`). Driver `org.postgresql:postgresql`.

RNF06 Cache Distribuído `infrastructure.cache.adapter + infrastructure.config.CacheConfig`
`RedisCacheAdapter` (usa `RedisTemplate<String, String>`). Aplicado no `ResolveUrlService`
via `Cache-Aside`.

RNF07 Observabilidade (Logs/Health) `infrastructure.observability` e `actuator endpoints`
`CustomHealthIndicator` (checa Redis/DB). Logs estruturados com `@Slf4j`. Dependência
`spring-boot-starter-actuator`.

RNF08 Documentação (OpenAPI) `infrastructure.config.OpenApiConfig` + dependência
`springdoc-openapi` `@OpenAPIDefinition` com info do sistema. Swagger UI acessível via
`/swagger-ui.html`.

RNF09 Configuração (12-factor) Diretório `resources/` + variáveis de ambiente `application.yml`
define default (ex: `spring.datasource.url: ${DB_URL:jdbc:postgresql://localhost:5432/urls}`).
Spring Boot lê `SPRING_*` envs.

3. Dependências Essenciais (pom.xml do módulo pai / módulos)

Atentando-se à Blocklist (vetado: Kafka, NoSQL, Filtros de Bloom) e evitando
overengineering, eis as dependências obrigatórias:

```
<!-- Parent: spring-boot-starter-parent (v3.2.x) -->
```

```
<dependencies>
```

```
  <!-- Core / Web (API) -->
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-web</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-validation</artifactId>
```

```
  </dependency>
```

```
  <!-- RNF05 - Persistência Relacional (PostgreSQL) -->
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>org.postgresql</groupId>
```

```
    <artifactId>postgresql</artifactId>
```

```
    <scope>runtime</scope>
```

```
  </dependency>
```

```
  <!-- RNF06 - Cache Distribuído (Redis - Lettuce é o default e mais performático) -->
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-pool2</artifactId> <!-- Para pooling de conexões Redis -->
    </dependency>

    <!-- RNF02 - Broker de Mensagens (RabbitMQ, compatível com SQS via Spring Cloud
AWS) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <!-- Alternativa para SQS (descomente se preferir AWS): -->
    <!-- <dependency>
        <groupId>io.awspring.cloud</groupId>
        <artifactId>spring-cloud-aws-starter-sqs</artifactId>
        <version>3.1.0</version>
    </dependency> -->

    <!-- RNF01 - Resiliência (Retry com Backoff) -->
    <dependency>
        <groupId>org.springframework.retry</groupId>
        <artifactId>spring-retry</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-aop</artifactId> <!-- Necessário para @Retryable -->
    </dependency>

    <!-- RNF07 - Observabilidade (Actuator + Micrometer) -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <!-- (Opcional) Métricas customizadas expostas para Prometheus -->
    <dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>

    <!-- RNF08 - Documentação OpenAPI/Swagger -->
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.5.0</version>
    </dependency>

```

```
<!-- Utilitários / Testes -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>
```

Justificativa de ausências: Não incluímos Spring Cloud Stream (para evitar abstrações desnecessárias sobre o broker, já que usaremos a API nativa do RabbitMQ/SQS) nem Hibernate Envers (auditoria) para não inflar o escopo. O pooling do Redis via commons-pool2 garante eficiência sem complexidade extra

4. Observações Finais sobre a Estrutura

Injeção de Dependências: A api e a worker compartilham as implementações de infraestrutura. O Worker ativa o `@RabbitListener` anotado no `UrlCreatedEventConsumer`, consumindo da fila configurada no `MessagingConfig`.

RNF01 na prática: O retry é aplicado exclusivamente nos adaptadores de repositório. Eventuais falhas no broker não são retentadas no escopo do retry de DB para não misturar responsabilidades; para o broker, utilizamos Dead Letter Queue (DLQ) nativa do RabbitMQ/SQS como complemento (não explicitado em detalhes, mas suportado pela estrutura).

Profile 12-Factor: O arquivo `application.yml` na API contém os placeholders `${...}`, garantindo que a imagem Docker seja genérica e as configs injetadas via variáveis de ambiente no orquestrador (Kubernetes/ECS).

Esta estrutura entrega um sistema robusto, testável (graças ao isolamento de portas) e alinhado com todas as restrições de resiliência, assincronicidade e cache exigidas, sem violar a Blocklist ou cair em overengineering.